

POLYLINE SAC

Documentación del Sistema

Sistema de Planificación y Gestión de Proyectos — Frontend

Frontend desarrollado con React 19, Vite 8 y Tailwind CSS v4

Septiembre 2026

Contenido

- **I. Introducción**
 - 1.1 Propósito del sistema
 - 1.2 Alcance del documento
- **II. Stack Tecnológico**
- **III. Arquitectura de la Aplicación**
 - 3.1 Patrón arquitectónico y separación de capas
 - 3.2 Características arquitectónicas y reactividad
 - 3.3 Flujo de autenticación y control de acceso (RBAC)
- **IV. Estructura del Proyecto**
 - 4.1 Árbol de directorios principal
 - 4.2 Descripción de carpetas y módulos clave
- **V. Mapa de Rutas**
 - 5.1 Catálogo consolidado de rutas del sistema
 - 5.2 Políticas de acceso, guards y navegación por roles
- **VI. Módulos Funcionales**
 - 6.1 Autenticación (Login)
 - 6.2 Panel de Control Central (Dashboard)
 - 6.3 Planificación Estratégica (Calendario Maestro e Hitos)
 - 6.4 Portafolio de Proyectos
 - 6.5 Gestión Operativa
 - 6.6 Equipo Técnico
 - 6.7 Recursos Humanos
 - 6.8 Integraciones Externas (Make, Drive, Revit)
 - 6.9 Gestión de Roles y Permisos (RBAC)
 - 6.10 Configuración y Personalización
- **VII. Controladores y Manejo de Estado**
 - 7.1 Controlador Principal (`useAppController`)
 - 7.2 Configuración Declarativa y Guards (`navigation.config.js`)
 - 7.3 Gestión de Datos y Estado Local
- **VIII. Integración con la API REST**
 - 8.1 Patrón de integración y configuración de Axios
 - 8.2 Catálogo consolidado de endpoints REST consumidos
 - 8.3 Clientes de servicio especializados

- **IX. Interfaz de Usuario y Diseño**
 - 9.1 Sistema Multitema Ergonómico
 - 9.2 Tipografía y Escala Ajustable (85% a 120%)
 - 9.3 Arquitectura de Layout y Componentes Estructurales
 - **X. Configuración de Entornos**
 - 10.1 Requisitos previos
 - 10.2 Instalación y ejecución local
 - 10.3 Scripts npm disponibles
 - 10.4 Compilación para producción
 - 10.5 Despliegue y Reglas de Reescritura SPA
 - **XI. Seguridad**
 - 11.1 Modelo RBAC y Protección de Vistas
 - 11.2 Manejo de Sesión y Almacenamiento Local
 - 11.3 Sanitización y Manejo de Errores
 - **XII. Pruebas**
 - 12.1 Estrategia y Framework de Pruebas
 - 12.2 Cobertura de Suites de Pruebas Automatizadas
 - 12.3 Ejecución de Pruebas
 - **XIII. Consideraciones Técnicas y Mejoras Futuras**
 - 13.1 Migración Plena a Endpoints REST de Django
 - 13.2 Visualización BIM / Revit 3D Nativa
 - 13.3 Integración Directa con Google Drive Picker
 - 13.4 Cobertura de Pruebas End-to-End (E2E)
 - **XIV. Glosario**
-

I. Introducción

El presente documento describe la arquitectura, componentes, fundamentos técnicos, catálogo de rutas, integración con servicios backend y procedimientos operativos del frontend del sistema **Project Planner**. Esta aplicación constituye la capa de presentación e interacción con el usuario de una plataforma integral orientada a la planificación estratégica corporativa, supervisión de portafolios de proyectos, coordinación de operaciones técnicas en campo y gestión del talento humano en empresas de ingeniería, construcción y consultoría técnica.

El proyecto, identificado técnicamente como `project-planner-frontend`, se desarrolla como una Single Page Application (SPA) moderna de alto rendimiento implementada en React 19 y empaquetada con Vite 8. Se comunica mediante peticiones asíncronas REST con un backend de servicios de misión crítica construido en Django REST Framework y respaldado por una base de datos relacional PostgreSQL.

1.1 Propósito del sistema

El sistema Project Planner permite a las diferentes áreas de la organización:

- **Panel de Control Central (Dashboard):** Visualizar indicadores clave de desempeño (KPIs), métricas consolidadas, resumen financiero y accesos rápidos a proyectos e hitos críticos.
- **Planificación Estratégica:** Administrar cronogramas maestros anuales mediante diagramas de Gantt interactivos, hitos críticos, tolerancias y objetivos corporativos.

- **Portafolio de Proyectos:** Supervisar avances porcentuales reales, presupuestos asignados frente a ejecutados, estados de ciclo de vida y asignación de líderes técnicos.
- **Gestión Operativa:** Coordinar despachos técnicos, órdenes de trabajo, tareas en campo y optimización de tiempos con algoritmos de ruta crítica.
- **Equipo Técnico:** Administrar especialistas organizados por área técnica especializada (Sistemas, Civil, Arquitectura y Estructuras).
- **Recursos Humanos:** Controlar la nómina de colaboradores, estado de contratos, disponibilidad operativa (Activo, Stand-by, Apoyo) y perfiles profesionales.
- **Integraciones Externas:** Monitorear y gestionar enlaces de automatización con Make (Integromat), almacenamiento en Google Drive y modelos BIM en Autodesk Revit.
- **Control de Acceso y Roles (RBAC):** Delimitar funciones confidenciales y administrativas exclusivas para perfiles de Project Manager frente a especialistas técnicos.
- **Personalización Multitema:** Adaptar la interfaz gráfica mediante tres esquemas visuales ergonómicos (Claro, Oscuro y Medianoche) y escalamiento tipográfico dinámico.

1.2 Alcance del documento

Este documento cubre de manera integral la capa frontend de la solución: fundamentación técnica, stack de dependencias, arquitectura de software, mapa de rutas, controladores de estado, integración con la API REST, interfaces de usuario, configuración de entornos, directrices de despliegue, políticas de seguridad y suites de pruebas automatizadas.

II. Stack Tecnológico

La aplicación está construida sobre tecnologías de última generación del ecosistema JavaScript/React, priorizando reactividad, velocidad de carga, mantenibilidad y accesibilidad ergonómica.

Tecnología	Versión	Propósito en la Solución
React	19.2.x	Biblioteca declarativa principal para la construcción de interfaces mediante componentes funcionales.
Vite	8.2.x	Herramienta de compilación y empaquetado ultrarrápido con Hot Module Replacement (HMR).
Tailwind CSS	4.3.x	Framework utilitario de estilos con directivas nativas y soporte para temas dinámicos.
React Router DOM	7.18.x	Enrutamiento declarativo del lado del cliente (SPA) y sincronización con History API.
Axios	1.19.x	Cliente HTTP basado en promesas con soporte de interceptores para comunicación con Django REST.
Lucide React	1.34.x	Catálogo iconográfico vectorial estilizado, accesible y altamente optimizado.
Oxlint	1.79.x	Linter de código estático de alto rendimiento implementado en Rust para asegurar calidad de código.
Node.js Test Runner	20+ nativo	Ejecutor de pruebas automatizadas sin dependencias externas pesadas para lógica y componentes.

III. Arquitectura de la Aplicación

3.1 Patrón arquitectónico y separación de capas

La aplicación adopta una arquitectura desacoplada por capas y dominios funcionales, garantizando una clara separación de responsabilidades:

- **Capa de Presentación (Views):** Componentes visuales organizados por dominio (`admin/` , `technical/` , `human-resources/` , `common/` , `auth/`). Cada vista es una función pura que recibe datos y callbacks a través de props.
- **Capa de Control y Orquestación (Controllers):** Centralizada en custom hooks como `useAppController.jsx` . Maneja el ciclo de vida de la aplicación, estado de autenticación, resolución de roles y navegación reactiva.
- **Capa de Servicios y Red (Services):** Abstracciones modulares (`api.js` , `projectsApi.js` , `makeApi.js` , `notificationsApi.js`) que encapsulan las llamadas REST a Django mediante Axios.
- **Capa de Configuración y Guards (Config):** Definición centralizada de navegación, roles permitidos, metadatos y funciones de guardia (`hasViewAccess`).
- **Capa de Datos Simulados y Adaptadores (Mocks):** Conjuntos de datos estructurados para desarrollo aislado y pruebas unitarias deterministas.

3.2 Características arquitectónicas y reactividad

- **Arquitectura SPA Pura:** Renderizado del lado del cliente sin recargas completas de página, proporcionando transiciones fluidas e instantáneas.
- **Gestión de Estado Ligera:** Uso idiomático de React Hooks (`useState` , `useEffect` , `useCallback` , `useMemo`), evitando dependencias globales voluminosas como Redux.
- **Sincronización Bidireccional con URL:** La URL del navegador refleja exactamente la vista activa, permitiendo navegación con historial (Back/Forward) y marcadores directos.
- **Sistema de Temas sin FOUC:** El tema seleccionado (`light` , `dark` , `midnight`) se inyecta directamente como clase en el elemento raíz `<html>` , previniendo parpadeos de estilo.

3.3 Flujo de autenticación y control de acceso (RBAC)

El flujo de seguridad garantiza la integridad de las sesiones de usuario:

1. **Inicio de Sesión:** El usuario ingresa credenciales en `/login` . El frontend despacha una petición `POST /api/auth/login/` al backend.
2. **Validación de Rol:** El backend verifica las credenciales y valida la concordancia con el rol seleccionado (Project Manager vs Equipo Técnico).
3. **Persistencia de Sesión:** Al recibir respuesta afirmativa (`200 OK`), los datos del usuario se almacenan en `localStorage` bajo la clave `project_planner_user` .
4. **Protección de Vistas:** Cada intento de acceso a una ruta pasa por la guardia `hasViewAccess()` . Si un usuario técnico intenta acceder a rutas administrativas confidenciales (ej. `/roles`), es redirigido automáticamente a `/dashboard` .
5. **Cierre de Sesión:** La función `handleLogout` elimina la sesión local y notifica al backend vía `POST /api/auth/logout/` para registrar la desconexión.

IV. Estructura del Proyecto

4.1 Árbol de directorios principal

```

frontend/
├─ package.json          # Metadatos, scripts npm y dependencias
├─ vite.config.js        # Configuración de compilación Vite y plugin Tailwind
├─ index.html            # Documento HTML raíz y punto de montaje DOM
├─ src/
│   └─ main.jsx          # Entrada JavaScript de montaje React
│   └─ App.jsx           # Layout maestro y orquestador de componentes
│   └─ index.css         # Directivas maestras Tailwind y temas
│   └─ assets/           # Logotipos corporativos y assets estáticos
│   └─ components/layout/ # Componentes estructurales permanentes
│       └─ Sidebar.jsx   # Menú lateral colapsable con enlaces filtrados
│       └─ Topbar.jsx    # Barra superior con buscador y notificaciones
│       └─ MobileNavigation.jsx # Barra de navegación móvil inferior
│       └─ ProfileControls.jsx # Dropdown de perfil, roles y logout
│   └─ config/
│       └─ navigation.config.js # Catálogo único de rutas, roles y guards
│   └─ controllers/
│       └─ index.js       # Barrel de controladores
│       └─ useAppController.jsx # Hook principal de estado y navegación
│   └─ views/            # Vistas agrupadas por dominio funcional
│       └─ index.js       # Barrel unificado de vistas
│       └─ auth/          # LoginScreen y LoadingScreen
│       └─ admin/         # StrategicPlanning y RolesManagement
│       └─ technical/     # Portfolio, Operations y TechnicalTeam
│       └─ human-resources/ # HumanResources
│       └─ common/        # DashboardContent, Integrations y Configuration
│   └─ mocks/            # Generadores de datos simulados por dominio
│   └─ services/         # Clientes de API REST (Axios)
│       └─ api.js         # Instancia central Axios con baseURL dinámica
│       └─ projectsApi.js # Servicios para portafolio de proyectos
│       └─ makeApi.js     # Servicios de integración con Make
│       └─ notificationsApi.js # Servicios de alertas y notificaciones
│   └─ tests/            # Suites de pruebas con Node.js Test Runner

```

4.2 Descripción de carpetas y módulos clave

- `src/config/` : Contiene `navigation.config.js` , constituyendo la **única fuente de verdad** para identificadores de ruta, etiquetas, iconos vectoriales, roles admitidos y funciones de guardia.
- `src/controllers/` : Aloja el hook `useAppController.jsx` , desacoplando completamente la lógica de negocio, autenticación, filtrado de búsquedas y navegación de los componentes visuales.
- `src/views/` : Agrupa las pantallas por dominio de negocio (`admin` , `technical` , `human-resources` , `common`), facilitando el mantenimiento y la escalabilidad modular.
- `src/services/` : Centraliza el cliente HTTP Axios preconfigurado con tiempos de espera, cabeceras automáticas y manejo estandarizado de errores REST.
- `src/mocks/` : Funciones generadoras puras que proporcionan datos iniciales y permiten ejecutar pruebas deterministas sin depender de la base de datos viva.

V. Mapa de Rutas

5.1 Catálogo consolidado de rutas del sistema

Todas las rutas autenticadas son administradas de forma declarativa. La correspondencia entre URLs, componentes y niveles de acceso es la siguiente:

Ruta URL	ID Vista	Componente	Dominio	Roles Permitidos	Menú	Propósito Funcional
/login	login	LoginScreen	Auth	Público	No	Autenticación con credenciales y selector de perfil.
/o /dashboard	dashboard	DashboardContent	Común	Admin, User	Sí	Panel general con indicadores clave, métricas e hitos.
/planning	planning	StrategicPlanning	Admin	Admin, User*	Sí	Planificación anual, Gan maestro y crítica.
/portfolio	portfolio	Portfolio	Técnico	Admin, User	Sí	Supervisión de portafolio de proyectos y avances reales.
/operations	operations	Operations	Técnico	Admin, User	Sí	Gestión operativa de órdenes de trabajo y tareas en campo.
/technical-team	technical-team	TechnicalTeam	Técnico	Admin, User	Sí	Directorio de especialistas técnicos por especialidad.
/human-resources	human-resources	HumanResources	RR. HH.	Admin, User	Sí	Gestión de personal de planta, disponibilidad y contratos.
/integrations	integrations	Integrations	Común	Admin, User	Sí	Monitoreo y diagnóstico de conectores Make, Drive y Revit.
/roles	roles	RolesManagement	Admin	Solo Admin	No	Administración de confianza de usuarios y permisos RBAC.
/configuracion	configuracion	Configuration	Común	Admin, User	No	Ajustes de cuenta, preferencias.

Ruta URL	ID Vista	Componente	Dominio	Roles Permitidos	Menú	Propósito Funcional
						selector de temas.

Nota: En `/planning`, los usuarios técnicos acceden en modo de solo lectura (`canManage = false`).

5.2 Políticas de acceso, guards y navegación por roles

- **Filtrado Dinámico de Menús:** Las funciones `getAllowedSidebarItems(user)` y `getAllowedMobileItems(user)` evalúan en tiempo de ejecución las opciones de menú lateral y móvil según los privilegios del usuario autenticado.
- **Ruta por Defecto (Fallback de Enrutamiento):** Cualquier ruta no reconocida o la raíz `/` es redirigida de forma transparente hacia `/dashboard` mediante `getViewIdFromPath()`.
- **Búsqueda Contextual Dinámica:** Al navegar entre módulos, el buscador de la barra superior actualiza automáticamente su texto de ayuda mediante `getSearchPlaceholder(activeView)`.

VI. Módulos Funcionales

6.1 Autenticación (Login)

Formulario de acceso con selector de perfil (Project Manager o Equipo Técnico), validación de correo electrónico y contraseña. Al autenticar exitosamente, sincroniza la sesión con el backend y redirige al panel de control. Incluye mensajes de error contextuales ante credenciales inválidas o cuenta inactiva.

6.2 Panel de Control Central (Dashboard)

Presenta un panorama ejecutivo en tiempo real: total de proyectos en curso, presupuesto consolidado, distribución de tareas operativas e hitos con vencimiento próximo. Permite navegación directa a los módulos correspondientes.

6.3 Planificación Estratégica (Strategic Planning)

Centro neurálgico de supervisión corporativa anual:

- **Calendario Maestro (Gantt Anual):** Visualización en 12 columnas mensuales con barras proporcionales según `start_date` y `duration_months`.
- **Hitos Globales:** Gestión de entregas críticas con estados visuales (Pendiente, Próximo, Cumplido, Atrasado) e identificación de hitos bloqueantes de ruta crítica.
- **Ajuste de Calendario (`AdjustScheduleDialog`):** Modificación de mes de inicio y duración con vista previa interactiva y registro obligatorio del motivo de reajuste en historial de auditoría.
- **Control Presupuestario:** Visualización en gráfico de dona del presupuesto global distribuido en disponible, ejecutado y comprometido.

6.4 Portafolio de Proyectos (Portfolio)

Administración integral de los proyectos de la organización:

- Catálogo en tarjetas con código corporativo (`PRJ-2026-XXX`), avance porcentual ponderado, presupuesto total y ejecutado, fechas límite y avatar del líder técnico.

- Modal de alta de nuevo proyecto con cálculo automatizado de duración y sincronización directa con PostgreSQL vía `projectsApi.js`.
- Filtros rápidos por estado (En Planificación, Activo, Pausado, Completado, Riesgo) y búsqueda en vivo.

6.5 Gestión Operativa (Operations)

Supervisión de despachos y órdenes de trabajo técnicas:

- Tablero de tareas con avance individual, duración en días, especialista asignado y categoría técnica.
- Identificación de tareas en Ruta Crítica y aplicación de márgenes de holgura/tolerancia de 7 días.

6.6 Equipo Técnico (Technical Team)

Directorio de especialistas técnicos agrupados por área de especialización:

- Clasificación en Sistemas, Ingeniería Civil, Arquitectura y Estructuras.
- Visualización de proyectos asignados, carga operativa y estado de disponibilidad en tiempo real.

6.7 Recursos Humanos (Human Resources)

Administración del talento y colaboradores de la empresa:

- Registro de colaboradores con modalidad contractual, cargo y datos de contacto.
- Estados de disponibilidad operativa: Activo, Stand-by o Apoyo temporal.

6.8 Integraciones Externas (Integrations)

Monitoreo de enlaces con plataformas y servicios de terceros:

- **Make (Integromat):** Conector con webhooks para despacho de eventos hacia Gmail y Discord, con botón de prueba de diagnóstico en vivo.
- **Google Drive API:** Enlaces a planos, memorias de cálculo y carpetas técnicas asociadas a tareas.
- **Revit BIM:** Sincronización de metadatos de modelos 3D y planos estructurales.

6.9 Gestión de Roles y Permisos (Roles Management)

Módulo confidencial accesible únicamente por administradores principales:

- Visualización de usuarios registrados en PostgreSQL, correo electrónico, rol y estado de conexión en vivo (`isOnline`).
- Asignación y revocación dinámica de permisos de SubAdministrador.

6.10 Configuración del Sistema (Configuration)

Ajustes personalizados de la experiencia de usuario:

- Selector de temas visuales: Claro, Oscuro y Medianoche.
- Control deslizante de escala tipográfica (desde 85% hasta 120%).
- Resumen de perfil y credenciales de acceso.

VII. Controladores y Manejo de Estado

7.1 Controlador Principal (`useAppController`)

El hook `useAppController.jsx` actúa como el director de orquesta de la aplicación:

- Gestiona los estados de autenticación (`currentUser` , `isAuthenticated`).
- Sincroniza la URL del navegador mediante `useLocation` y `useNavigate` .
- Controla los estados de datos y consultas de búsqueda para cada módulo.
- Calcula de manera memorizada (`useMemo`) el usuario visible y sus facultades de gestión (`canManage = isAdmin || isSubAdmin`).

7.2 Configuración Declarativa y Guards (`navigation.config.js`)

Centraliza las políticas de navegación en un solo archivo:

- `hasViewAccess(viewId, user)` : Determina si un perfil tiene autorización para visualizar una vista determinada.
- `getAllowedSidebarItems(user)` / `getAllowedMobileItems(user)` : Entregan los elementos de menú autorizados para renderizado visual.

7.3 Separación de Mocks y Servicios

La arquitectura permite alternar de forma transparente entre datos simulados estructurados (`src/mocks/`) y llamadas REST activas (`src/services/`), facilitando el desarrollo continuo y pruebas automatizadas.

VIII. Integración con la API REST

8.1 Patrón de integración y configuración de Axios

El cliente centralizado en `src/services/api.js` está construido con Axios:

- Base URL dinámica configurada mediante la variable de entorno `VITE_API_BASE_URL` (por defecto `http://localhost:8000/api`).
- Cabeceras estandarizadas `Content-Type: application/json` .
- Tiempos de espera configurados a 10 segundos para prevenir bloqueos de interfaz.

8.2 Catálogo consolidado de endpoints REST consumidos

Método HTTP	Endpoint Django REST	Descripción Técnica	Módulo Frontend
POST	<code>/api/auth/login/</code>	Autenticación de usuarios y validación de roles	Login
GET	<code>/api/auth/users-status/</code>	Consulta de usuarios con estado de conexión en vivo	Roles / Topbar
PATCH	<code>/api/auth/users-status/</code>	Modificación de privilegios de subadministrador	Roles
POST	<code>/api/auth/logout/</code>	Cierre de sesión y registro de desconexión	Topbar
GET	<code>/api/notifications/</code>	Alertas de asignación de proyectos en campana	Topbar
GET	<code>/api/projects/</code>	Listado completo de proyectos con progreso y líder	Portafolio / Planning

Método HTTP	Endpoint Django REST	Descripción Técnica	Módulo Frontend
POST	/api/projects/	Alta de nuevo proyecto en base de datos PostgreSQL	Portafolio
PATCH	/api/projects/:id/	Modificación de presupuesto, fechas o estado	Portafolio / Planning
POST	/api/projects/:id/optimize-schedule/	Cálculo de cronograma optimizado y ruta crítica	Operaciones
GET	/api/milestones/	Listado de hitos globales por proyecto	Planning / Dashboard
POST	/api/milestones/	Creación de nuevo hito crítico con validador	Planning
PATCH	/api/milestones/:id/	Actualización de estado y fecha límite de hito	Planning
GET	/api/tasks/	Listado de tareas operativas y ruta crítica	Operaciones
GET	/api/team-members/	Directorio de personal técnico y líderes disponibles	Equipo / Portafolio
GET	/api/technical-areas/	Catálogo de áreas técnicas especializadas	Equipo / Operaciones
GET	/api/integrations/make/status/	Estado de configuración del conector Make	Integraciones
POST	/api/integrations/make/test/	Diagnóstico de conexión en vivo con webhook Make	Integraciones
POST	/api/integrations/make/trigger/	Emisión programática de eventos a Make	Integraciones

8.3 Clientes de servicio especializados

- `projectsApi.js`: Métodos `getProjects()`, `createProject()`, `updateProject()`, `getProjectById()`.
- `makeApi.js`: Métodos `getMakeStatus()`, `testMakeConnection()`, `triggerMakeEvent()`.
- `notificationsApi.js`: Métodos `getNotifications()`, `markAsRead()`.

IX. Interfaz de Usuario y Diseño

9.1 Sistema Multitema Ergonómico

El sistema ofrece tres esquemas visuales diseñados para reducir la fatiga cognitiva y visual:

- **Tema Claro (Light)**: Fondo `#F8FAFC`, tarjetas `#FFFFFF`, bordes `#E2E8F0` y tipografía `#0F172A`. Ideal para entornos iluminados.
- **Tema Oscuro (Dark)**: Fondo `#0D1117`, tarjetas `#161B22`, bordes `#30363D` y tipografía `#F0F6FC`. Inspirado en la interfaz de GitHub Pro.
- **Tema Medianoche (Midnight)**: Fondo `#050B14`, tarjetas `#0A1120`, bordes `#164E63` y acentos `#06B6D4` (Cyan). Orientado a trabajo técnico prolongado.

9.2 Tipografía y Escala Ajustable (85% a 120%)

El usuario puede ajustar la densidad de la interfaz desde el panel de Configuración:

- Rango de 85% a 120% con persistencia en `localStorage`.
- Se aplica mediante factor de zoom CSS en el contenedor maestro sin romper la responsividad.

9.3 Layout Adaptativo y Componentes Estructurales

- **Sidebar:** Barra lateral colapsable para pantallas de escritorio con navegación filtrada por rol.
- **Topbar:** Encabezado con buscador contextual reactivo, selector de tema rápido, campana de notificaciones con badge de conteo y dropdown de perfil.
- **MobileNavigation:** Barra inferior optimizada para pantallas táctiles con botones ergonómicos y etiquetas compactas.

X. Configuración de Entornos

10.1 Requisitos previos

- Node.js versión 18.x o superior (recomendado Node.js 20+ LTS).
- npm versión 9.x o superior.
- Backend Django REST Framework en ejecución (puerto 8000 en entorno local).
- Base de datos PostgreSQL activa con migraciones ejecutadas.

10.2 Instalación y ejecución local

```
# 1. Clonar el repositorio y acceder a la carpeta frontend
cd project-planner/frontend

# 2. Instalar dependencias del proyecto
npm install

# 3. Configurar variables de entorno si es necesario (.env)
# VITE_API_BASE_URL=http://localhost:8000/api

# 4. Iniciar servidor de desarrollo con Vite
npm run dev

# 5. Abrir navegador en http://localhost:5173
```

10.3 Scripts npm disponibles

Script	Comando Ejecutado	Propósito Técnico
dev	vite	Servidor de desarrollo con recarga rápida (HMR).
build	vite build	Compilación optimizada para producción en carpeta <code>dist/</code> .
lint	oxlint	Análisis estático de calidad de código ultrarrápido.
preview	vite preview	Previsualización local del artefacto compilado en producción.

Script	Comando Ejecutado	Propósito Técnico
test:planning	node --test tests/strategicPlanning.test.mjs	Suite de pruebas unitarias de Planificación Estratégica.
test:portfolio	node --test tests/portfolio.test.mjs	Suite de pruebas del módulo Portafolio de Proyectos.
test:operations	node --test tests/operations.test.mjs	Suite de pruebas del módulo Gestión Operativa.
test:team	node --test tests/technicalTeam.test.mjs	Suite de pruebas del módulo Directorio Técnico.
test:hr	node --test tests/humanResources.test.mjs	Suite de pruebas del módulo Recursos Humanos.
test:integrations	node --test tests/integrations.test.mjs	Suite de pruebas del módulo Integraciones Externas.

10.4 Compilación para producción

Al ejecutar `npm run build`:

- Vite compila y minifica todos los componentes React y estilos Tailwind CSS.
- Aplica *tree-shaking* para remover código muerto y reducir el peso de los bundles.
- Genera nombres con hash criptográfico (ej. `app-a1b2c3d4.js`) para invalidación automática de caché en CDNs.
- El resultado se deposita en la carpeta `dist/`.

10.5 Despliegue y Reglas de Reescritura SPA

Al ser una aplicación de una sola página con enrutamiento del lado del cliente, el servidor web (Nginx, Vercel, Render o Cloudflare Pages) debe redirigir cualquier petición interna (`/portfolio` , `/planning` , `/operations`) hacia `/index.html` con código HTTP 200 para evitar errores 404.

XI. Seguridad

11.1 Modelo RBAC y Protección de Vistas

El sistema implementa control de acceso basado en roles con dos niveles fundamentales:

- **Project Manager (Admin):** Acceso total a todos los módulos, reprogramación de calendarios en Gantt, creación de hitos, alta de proyectos y gestión confidencial de roles de usuario.
- **Equipo Técnico (User):** Acceso operativo y de consulta. Puede visualizar proyectos e hitos, pero tiene deshabilitados los controles de edición y el módulo `/roles`.

11.2 Manejo de Sesión y Almacenamiento Local

- Los datos de usuario se almacenan de forma estructurada en `localStorage`.
- El sistema cuenta con mecanismos automáticos de sanitización y depuración al detectar discrepancias o estructuras obsoletas, forzando la reautenticación limpia.

11.3 Sanitización y Manejo de Errores

- Todos los formularios implementan validación de tipos, rangos de fechas válidas y límites de caracteres antes de despachar peticiones de red.
 - Los errores HTTP (400, 401, 403, 500) son interceptados y presentados al usuario mediante alertas amigables y accesibles.
-

XII. Pruebas

12.1 Estrategia y Framework de Pruebas

Project Planner Frontend utiliza el **Node.js Test Runner** nativo (`node:test` y `node:assert/strict`), complementado con Vite SSR (`createServer`) y `react-dom/server` para evaluar el renderizado estático de componentes sin necesidad de instalar entornos emulados pesados como JSDOM o Jest.

12.2 Cobertura de Suites de Pruebas Automatizadas

- **Planificación Estratégica (`strategicPlanning.test.mjs`):** Valida la carga del calendario maestro, cálculo de periodos de duración, validaciones de hitos globales, ajuste de proyectos con historial de auditoría y renderizado condicional según privilegios (`canManage = true/false`).
 - **Navegación y Permisos:** Valida que las funciones `hasViewAccess` , `getAllowedSidebarItems` y `getAllowedMobileItems` restrinjan adecuadamente los accesos según el rol.
 - **Módulos Técnicos y Operativos:** Pruebas unitarias dedicadas para portafolio, operaciones, equipo técnico, recursos humanos e integraciones.
-

XIII. Consideraciones Técnicas y Mejoras Futuras

13.1 Migración Plena a Endpoints REST de Django

Actualmente, los módulos de Portafolio y Make ya operan con endpoints vivos de Django y PostgreSQL. Como parte de la hoja de ruta inmediata, se completará la sustitución de los mocks restantes en Planificación Estratégica (`milestonesApi.js`), Operaciones Técnicas y Recursos Humanos.

13.2 Visualización BIM / Revit 3D Nativa

Integración de un visor 3D embebido basado en Autodesk Platform Services (Forge Viewer) para inspeccionar modelos de Revit directamente desde las tareas del proyecto.

13.3 Integración Directa con Google Drive Picker

Implementación del diálogo nativo Google Picker API para adjuntar documentación técnica y memorias de cálculo directamente desde las cuentas corporativas de Google Workspace.

13.4 Cobertura de Pruebas End-to-End (E2E)

Incorporación de suites de pruebas de extremo a extremo utilizando Microsoft Playwright para simular flujos de usuario completos entre frontend y backend en entornos de integración continua (CI/CD).

XIV. Glosario

Término	Definición Técnica en el Sistema
SPA	<i>Single Page Application</i> : Aplicación web que carga un único documento HTML y actualiza la vista de forma reactiva sin recargar la página.
RBAC	<i>Role-Based Access Control</i> : Mecanismo de seguridad que restringe el acceso a funciones y vistas según el rol asignado al usuario.
Diagrama de Gantt	Representación gráfica de cronogramas anuales donde las barras ilustran la ventana de ejecución de cada proyecto.
Hito Global (Milestone)	Evento o entrega crítica de un proyecto que marca el cumplimiento de una fase fundamental.
Hito Bloqueante	Hito cuya no validación detiene la ejecución del cronograma y activa la ruta crítica.
Ruta Crítica	Secuencia de tareas prioritarias que determina la duración mínima total del proyecto sin margen de holgura.
Make (Integromat)	Plataforma de automatización en la nube (iPaaS) utilizada para emitir webhooks hacia Gmail y Discord.
Vite	Herramienta de compilación para desarrollo frontend que aprovecha módulos ES nativos del navegador.
SSR (Server-Side Rendering)	Renderizado en servidor utilizado en las suites de pruebas para validar componentes React estáticamente.
Tailwind CSS	Framework CSS utilitario para diseño ergonómico de interfaces de alta personalización.
History API	API del navegador para manipular el historial de navegación sin causar recargas completas.
CORS	<i>Cross-Origin Resource Sharing</i> : Mecanismo de seguridad HTTP que autoriza peticiones entre frontend y backend en distintos dominios/puertos.